

SEPARAÇÃO: O QUE É UNIVERSAL vs. O QUE É ESPECÍFICO

Data: 2026-08-23

Contexto: Clarificar quais componentes da Fábrica são replicáveis universalmente e quais são específicos do domínio de publicações.

Objetivo: Evitar cópia cega. Novo projeto copia APENAS universal, adapta ESPECÍFICO.

REGRA DE OURO

UNIVERSAL (copiar 100%):

Estrutura, padrões, infraestrutura, governança, economia de tokens

ESPECÍFICO (usar como template, reescrever):

Skills de domínio, comandos de criação, gates de conteúdo, templates de output

MATRIZ DE SEPARAÇÃO

UNIVERSAL – COPIAR INTEGRALMENTE

Componente	Por Quê	Copiar?
CLAUDE.md (Seção 0–7)	Estrutura de governança agnóstica	✓ 100%
Seção 0: Economia Severa	Aplica a QUALQUER escopo	✓ 100%
Seção 1: Regras Globais (R1–R17)	Princípios de qualidade universais	✓ ~80% (adaptar nomes/ exemplos)
Padrão de Skills	Frontmatter + fluxo estruturado	✓ 100%
5 Skills de Economia	caveman, headroom, lean-ctx, rtk-memory, pre-flight-check	✓ 100%
RTK.md	Guia de token economy	✓ 100%
Padrão de Comandos	Fases P&D → Manufatura → Review → Entrega	✓ 100%
Padrão de Subagentes	Atomização, pool, retentativa 3x, barreira	✓ 100%
Padrão de Scripts Python	Argparse, console_utf8(), JSON reports, idempotência, exit codes	✓ 100%
Padrão de MCPs	Registro declarativo, estrutura	✓ 100%
Portabilidade Multi-IDE	setup-links.ps1/.sh, junctions, hardlinks	✓ 100%
Pre-commit Hook	Validação pré-commit, exit code logic	✓ 100%
settings.json (hooks)	PostToolUse, SessionStart, estrutura	✓ 100%
Auto-memória	MEMORY.md, RTK-SCRATCHPAD.md	✓ 100%
.code-review-graph	Integração com MCP de grafo	✓ 100%

ESPECÍFICO – USAR COMO TEMPLATE, REESCREVER

EXEMPLOS: UNIVERSAL vs. ESPECÍFICO

Exemplo 1: Padrão de Skill (UNIVERSAL)

name: nome-skill
description: Uma frase do resultado

Skill_NomeCapitalizado

Você é o especialista em [domínio].

Regras

- REGRA 1, 2, 4 do CLAUDE.md

Padrão de Fluxo

Passo 1 – [Fase 1]

Passo 2 – [Fase 2]

Passo 3 – [Fase 3]

Checklist de Entrega

✅ Este padrão é 100% replicável. Mude apenas [domínio], [Fase 1–3], [Checklist].

Exemplo 2: Skill de Redação (ESPECÍFICO)

Fábrica:

name: redator-eita
description: Expandir draft estratégico em texto final EITA-V2

Skill_Redator_EITA

Você é o redator de manufatura final (EITA-V2).

Template Obrigatório

1. Introdução

2. Explica

3. Ilustra

4. Técnica

5. Aplica

6. Conclusão

7. Referências

✗ NÃO copie direto. Reescreva para seu domínio:

SaaS (exemplo):

name: feature-implementer
description: Implementar feature do design até testes

Skill_FeatureImplementer

Você é o implementador de features (Sprint).

```
## Template Obrigatório
## 1. Context (Por que existe)
## 2. Design (Arquitetura)
## 3. Implementation (Código)
## 4. Tests (Cobertura ≥90%)
## 5. Deployment (Rollout)
## 6. Monitoring (Métricas)
## 7. Rollback (Plano contingência)
```

Exemplo 3: Script Determinístico (UNIVERSAL PADRÃO)

Padrão (copiar):

```
#!/usr/bin/env python3
"""
<modulo>: <descricao>
Entrada: <input>
Saída: <output.json>
Garantias: idempotente, sem side effects, exit code 0 = sucesso
"""

def console_utf8():
    # UTF-8 everywhere
    ...

def main():
    relatorio = {"status": "falha", "erros": []}
    try:
        # sua lógica
        relatorio["status"] = "sucesso"
    finally:
        # gravar JSON sempre
        Path(f"output/{slug}/relatorio.json").write_text(...)
    return 0 if relatorio["status"] == "sucesso" else 1

if __name__ == "__main__":
    console_utf8()
    sys.exit(main())
```

✓ Este padrão é 100% replicável. Mude apenas [sua_logica].

Gate de Conteúdo (específico, template):

Fábrica:

```
# scripts/validar-referencias.py
# Valida URL/DOI reais, reprova 4xx/DNS
```

✗ NÃO copie direto. Reescreva para seu domínio:

SaaS (exemplo):

```
# scripts/validar-testes.py
# Valida cobertura ≥90%, reprova < threshold
```

BLUEPRINT DE REPLICAÇÃO (CORRIGIDO)

Passo 1: Copiar APENAS Universal (2 horas)

```
# Copiar estrutura completa (será filtrada depois)
cp -r fabrica novo-projeto
```

```
# Remover específico (será reescrito)
rm -rf novo-projeto/.claude/skills/pesquisador
rm -rf novo-projeto/.claude/skills/redator-*
rm -rf novo-projeto/.claude/skills/compilador-*
rm -rf novo-projeto/.claude/commands/criar-*
rm -rf novo-projeto/.claude/commands/compilar-*
rm -f novo-projeto/scripts/validar-referenc*
rm -f novo-projeto/scripts/gerar-cap*
rm -rf novo-projeto/templates/template_eita*
```

Reste universal: - ☒ .claude/CLAUDE.md (governança) - ☒ .claude/skills/
{caveman,headroom,lean-ctx,rtk-memory,pre-flight-check} - ☒ .claude/settings.json - ☒ .claude/
RTK.md - ☒ setup-links.ps1/.sh - ☒ Padrões de script Python - ☒ Padrão de MCP - ☒ .git/
hooks/pre-commit - ☒ scripts/pdf_typst.py (compilação genérica)

Passo 2: Reescrever Específico (4–8 horas)

O Que	Copia?	Reescreve?	Exemplo
Skills de domínio	Padrão de estrutura	100% (logica)	pesquisador → seu-especialista-1
Comandos de criação	Padrão de orquestração	100% (logica)	/criar-livro → /criar-feature
Gates de conteúdo	Padrão de script	100% (logica)	validar-referencias → validar-testes
Templates	Estrutura básica	100% (layout)	EITA-V2 → Seu formato
Framework	Conceitual	100% (sections)	7 seções EITA → N seções sua
tipos_obra.py	Padrão declarativo	100% (tipos)	{livro, tcc, artigo} → seus tipos
Compilação	Pandoc+Typst	Adaptar (~20%)	Seu pipeline de output

CHECKLIST: “QUAL CÓPIA?”

Pergunta: Devo copiar diretamente?

Componente	Pergunta	Resposta	Ação
CLAUDE.md	É independente de domínio?	SIM	✅ Copiar 100%
Skills economia	São específicas de redação?	NÃO	✅ Copiar 100%
Padrão de Skill	A estrutura muda por domínio?	NÃO	✅ Copiar 100% (mudar conteúdo)
Skill pesquisador	É específica de publicações?	SIM	❌ Reescrever para seu domínio
Comando /criar-livro	É específica de livros?	SIM	❌ Reescrever (/criar-feature, etc.)
validar-referencias.py	Aplica a URLs/DOIs?	SIM (específico)	❌ Reescrever (validar-testes, etc.)
template_eita.md	É específica de 7 seções?	SIM	❌ Reescrever seu template
setup-links.ps1	É independente de projeto?	SIM	✅ Copiar 100%
Pre-commit hook	É agnóstica?	SIM	✅ Copiar 100%
Token economy	Aplica a qualquer escopo?	SIM	✅ Copiar 100%

RESUMO FINAL

Copiar 100% (≈40% do projeto): - ✅ Governança (CLAUDE.md R0–R17) - ✅ Padrões (skills, commands, subagents, scripts, MCPs) - ✅ Token economy (5 skills + RTK) - ✅ Infraestrutura (junctions, hooks, CI/CD) - ✅ Portabilidade multi-IDE - ✅ Auto-memória (MEMORY.md, RTK-SCRATCHPAD)

Reescrever 100% (≈60% do projeto): - ❌ Skills de domínio (usar padrão, mudar lógica) - ❌ Comandos de criação (usar padrão, mudar workflow) - ❌ Gates de validação (usar padrão, mudar critérios) - ❌ Templates (usar estrutura, mudar formato/seções) - ❌ Compilação final (adaptar pipeline) - ❌ MCPs específicas (adaptar schema) - ❌ Squad especializada (seu time)

Resultado: Novo projeto nasce com **infraestrutura 100% pronta**, mas com **domínio 100% customizado**.

TEMPO ESTIMADO

Etapa	Tempo	O Quê
Clonar + setup	30 min	Scripts universal
Remover específico	30 min	Limpar arquivos da Fábrica
Customizar CLAUDE.md	30 min	Tema, squad, exemplos
Reescrever skills domínio	2-3h	3-5 skills especializadas
Reescrever comandos	2-3h	3-5 comandos orquestradores
Reescrever gates	1-2h	4-6 scripts de validação
Adaptar templates	1h	Output final
Testar e validar	1h	Checklist + pre-commit
Total	10-14 horas	Projeto pronto

EXEMPLO PRÁTICO: CLONANDO PARA SAAS

Copiar 100% (30 min)

```
cp -r fabrica novo-saas-project
cd novo-saas-project

# Manter (universal)
rm -rf .claude/skills/{pesquisador,redator-*,compilador-*,revisor-*}
rm -rf .claude/commands/{criar-livro,criar-tcc,criar-artigo,compilar-*,reescrever*}
rm -f scripts/{validar-referenc*,gerar-capa*,auditar-obra*}

# Restar:
# ✅ .claude/CLAUDE.md
# ✅ .claude/skills/{caveman,headroom,lean-ctx,rtk-memory,pre-flight-check}
# ✅ setup-links.ps1
# ✅ .git/hooks/pre-commit
# ✅ scripts/pool-capitulos.py (padrão de pool)
```

Reescrever (4-8h)

Skills SaaS:

```
.claude/skills/
├─ product-manager          (novo: visão + requisitos)
├─ architect                (novo: design + decisões)
├─ dev-senior               (novo: implementação)
├─ qa-engineer              (novo: testes)
└─ [economia] caveman, headroom, lean-ctx (✅ mantém)
```

Comandos SaaS:

```
.claude/commands/
├─ criar-feature            (novo: epic → feature → PR)
├─ debugar-issue            (novo: bug triage → fix → validation)
├─ refactor-seguro          (novo: teste-driven refactor)
└─ [economia] calcular-gastos-sessao (✅ mantém)
```

Scripts SaaS:

```
scripts/
├─ validar-testes.py        (novo: cobertura ≥90%)
└─ validar-performance.py   (novo: P99 latency)
```

|— validar-seguranca.py (novo: OWASP top 10)
|— [universal] pool-capitulos.py → pool-features.py (adaptar nomes)

CONCLUSÃO

Novo blueprint:

✓ **UNIVERSAL:** Copiar tudo (governança, padrões, economia, infraestrutura)

✗ **ESPECÍFICO:** Reescrever tudo (skills, commands, gates, templates, compilação)

Tempo real: 10–14 horas (não 4–6 semanas)

Resultado: Projeto novo com **infraestrutura profissional pronta, customizado para seu domínio.**